Lab 7: Fuzzing for Input Validation Bugs

Kishan S
SRN: PES2UG23CS277
Software Engineering Lab (5th Semester)

November 5, 2026

Objective

The aim of this lab was to use Hypothesis to fuzz three small functions from the bowling alley module, find the inputs that break them, and then fix the module so that no input can make it crash.

What I used

- Python 3.14.4
- Hypothesis 6.170.14
- pytest 9.2.0

The module is `bowling_alley_parser.py`. It has three functions: `format_player_name`, which title cases a name and cuts it to ten characters, `parse_frame`, which turns `X`, `7/` or `45` into pins, and `parse_scoreboard`, which turns `Asha:150,Ravi:132` into a dictionary. The test file is `test_bowling_alley_parser.py`, and the starter only had three empty test bodies with the `@given(st.text() | st.none())` decorator already written.

Step 2: running the buggy module

I ran `python bowling_alley_parser.py` first, with normal input, to see what the module is supposed to do. The run is shown in Figure 1. Everything looks fine here: the name comes out as `Asha`, the spare `7/` gives `(7, 3)`, and the scoreboard gives a dictionary with both players.

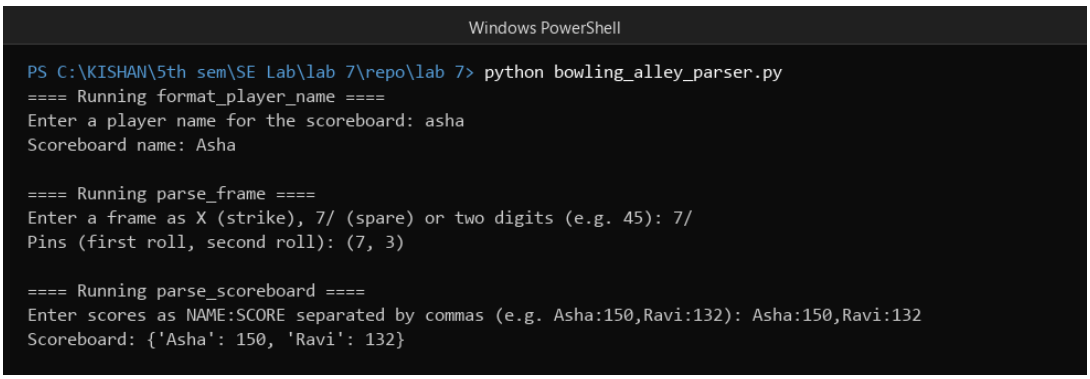


Figure 1: Running the buggy module with normal input. The happy path works.

The problem is that the module only checks nothing. The docstrings say it out loud: “Assumes name is a non-empty string”, “Assumes text is a valid frame”, “Assumes every entry is written as name:score”. Any input outside those assumptions raises an exception.

Step 3: writing the Hypothesis tests

I kept the decorator from the starter and filled the three bodies with the try and except shape from the handout, plus one check on the type of the result.

```
@given(st.text() | st.none())
def test_format_player_name_no_crash(s):
    # To-Do: Using try and except complete this function.
    try:
        result = format_player_name(s)
    except Exception as e:
        assert False, f"format_player_name crashed with: {e}"
    assert isinstance(result, str)
```

The other two tests have the same shape. The frame test also checks that the answer is a tuple of length two, and the scoreboard test checks that the answer is a dictionary.

Step 4: the tests on the buggy module

Figure 2 shows the run. All three tests fail. Hypothesis prints the smallest input that breaks each one, and it does the shrinking on its own.

The first one is easy to read. Hypothesis started from some random string, kept cutting it down, and ended on `None`. `None` has no `.strip()`, so the function dies with an `AttributeError`. The same thing happens in `parse_frame` and `parse_scoreboard`, but those two functions have more than one bug. Hypothesis reported **5 distinct failures** for `parse_frame` and **3 distinct failures** for `parse_scoreboard`. Table 1 lists all the inputs it printed.

Function	Input found	Error
`format_player_name`	`None`	`AttributeError: 'NoneType' object has no attribute 'strip'`
`parse_frame`	`None`	`AttributeError: 'NoneType' object has no attribute 'strip'`
`parse_frame`	`''`	`IndexError: string index out of range`
`parse_frame`	`'0'`	`IndexError: string index out of range`
`parse_frame`	`':'`	`ValueError: invalid literal for int() with base 10: ':'`
`parse_frame`	`'0:'`	`ValueError: invalid literal for int() with base 10: ':'`
`parse_scoreboard`	`None`	`AttributeError: 'NoneType' object has no attribute 'split'`
`parse_scoreboard`	`''`	`ValueError: not enough values to unpack (expected 2, got 1)`
`parse_scoreboard`	`':'`	`ValueError: invalid literal for int() with base 10: ''`

Table 1: Every failing input that Hypothesis shrank to, copied from the run in Figure 2.



Figure 2: pytest on the buggy module. Three tests fail, here is the first one with the shrunk input `s=None`.

Step 5: fixing the module

I fixed each function the same way. Check the type first, then check the length, then check the characters, and return a safe value instead of raising. For `parse_frame` the safe value is `(0, 0)`, for `parse_scoreboard` it is an empty dictionary, and for `format_player_name` it is an empty string. This is the new body of `parse_frame`.

```
if not isinstance(text, str):
    return 0, 0
text = text.strip().upper()
if text == 'X':
    return 10, 0
if len(text) != 2:
    return 0, 0
```

```
digits = '0123456789'
if text[0] not in digits:
    return 0, 0
first = int(text[0])
if text[1] == '/':
    if first == 10:
        return 0, 0
    return first, 10 - first
if text[1] not in digits:
    return 0, 0
second = int(text[1])
if first + second > 10:
    return 0, 0
return first, second
```

The length check removes the `IndexError` on `''` and `'0'`. The digit check removes the `ValueError` on `':'`. I also added two rules that the handout did not ask for but that follow from the game: a spare cannot start with 10 pins, and two rolls in one frame cannot add to more than 10.

`parse_scoreboard` skips a bad entry instead of failing the whole line, so one typo does not throw away the other players. `format_player_name` returns an empty string when the name is missing.

Running the fixed module and re-running the tests

Figure 3 shows the fixed module on the same kind of input, but with a bad frame and a broken score entry mixed in. Nothing crashes. The name is title cased, the bad frame gives `(0, 0)`, and the bad score entry is dropped while `Asha` and `Ravi` are kept.

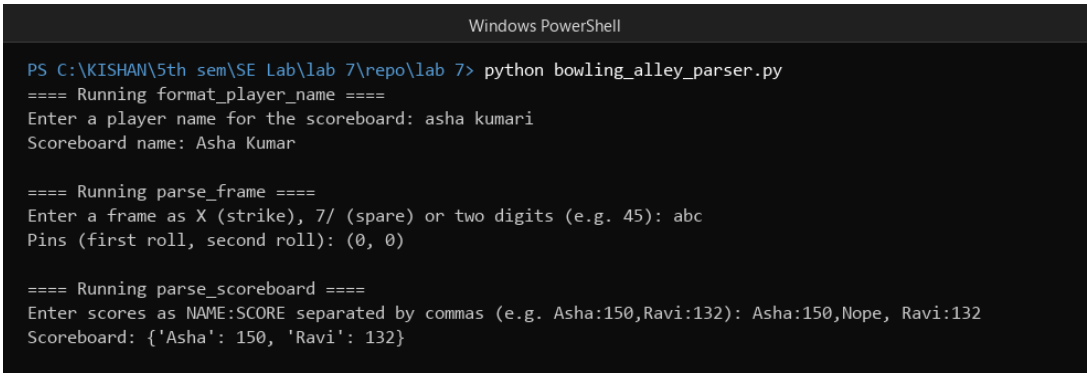


Figure 3: Running the fixed module with a bad frame and a bad score entry. It does not crash.

Figure 4 shows the second test run. All three tests pass.

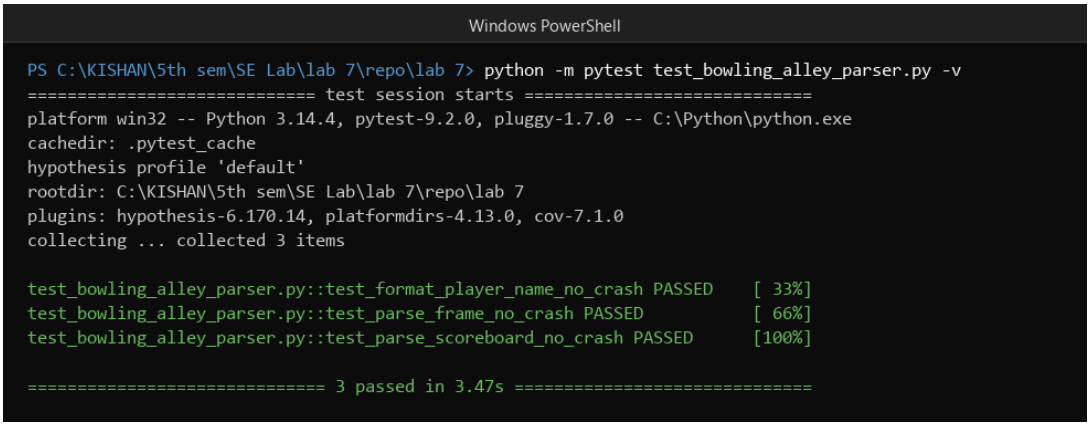


Figure 4: pytest on the fixed module. Three tests pass.

Run	Command	Result
Buggy module	`python -m pytest test_bowling_alley_parser.py -q`	3 failed in 2.78s
Fixed module	`python -m pytest test_bowling_alley_parser.py -v`	3 passed in 3.47s

Table 2: The two test runs, numbers copied from the terminal output.

Reflection

1. Which bugs did Hypothesis find that a hand written test would miss?

When I write tests by hand I think about the mistakes a user makes, like typing `one,two` instead of numbers. I do not think about `None` or about an empty string. `None` is not something a user types into the console, so it looked silly at first. But the same function is what a web form or a file reader will call, and there the missing value is common. The empty string was the best find. `parse_frame` reads `text[0]` and `text[1]` with no length check, so `''` and `'0'` both blow up with an `IndexError`. I would not have written that test.

2. Why does Hypothesis shrink the failing input, and how did that help?

The first input that fails is random and can be long, for example a 40 character string with junk in it. That is hard to read and hard to reason about. Hypothesis keeps trying smaller versions while the test still fails, so it ends on the smallest one. For this lab it ended on `None`, `''`, `'0'`, `':'` and `'0:'`, which are the small cases I needed. I put them straight into the bug table and then fixed the function for each one. Without shrinking I would have had to cut the string by hand and guess which character matters.

3. What did the fix change, and what can fuzzing still not check?

Every function now checks the type, the length and the characters before it does any work, and it returns a safe value instead of raising. This makes the module safe to call from any place that passes user input. But fuzzing only checks that the function does not crash. It cannot tell me if `(0, 0)` is the right answer for a bad frame, because the return type is still a tuple of two numbers. It also cannot check `main()`, since that needs a real keyboard. The tests check no crash and the return type only, so the actual pin numbers are still on me to verify.